

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)»**

**ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ИНФОРМАТИКИ
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

**Зачётная работа №2
по дисциплине «Алгоритмы и структуры данных»
на тему «ПОДДЕРЖКА ОБРАБОТКИ ИСКЛЮЧИТЕЛЬНЫХ
СИТУАЦИЙ»**

Выполнили студенты группы 3312

Лебедев И.А.

Шарапов И.Д.

Принял старший преподаватель Колинко П.Г.

Санкт-Петербург
2025

Содержание

Цель работы	3
Задание	3
Описание изменений.....	3
Описание классов исключений.....	4
Обоснование набора и вида классов исключений	5
Обоснование размещения операторов throw	5
Обоснование размещения блоков try-catch	6
Текст программы.....	7
Результаты работы программы.....	11
Выводы	12
Список используемых источников.....	12

Цель работы

Освоение принципов обработки исключительных ситуаций.

Задание

Модифицировать исходную программу, работающую с библиотекой фигур, добавив механизм обработки исключительных ситуаций. Реализовать выявление и генерацию различных типов ошибок, включая:

- некорректные параметры при создании или масштабировании фигуры;
- выход фигуры за пределы экрана при отрисовке.

Разработать и использовать собственные классы исключений, встроить *throw*-операторы в соответствующих местах кода. Продемонстрировать обработку не менее двух исключений разного уровня сложности с минимальным искажением итогового изображения.

Описание изменений

В обновлённой версии программы, реализующей работу с графическими фигурами, была добавлена полноценная поддержка обработки исключительных ситуаций. Основные изменения затронули следующие компоненты:

1. Реализация пользовательских классов исключений. В код были добавлены классы *NegativeSizeException* и *OutOfScreenException*, унаследованные от *std::exception*, что позволяет генерировать и перехватывать содержательные ошибки, возникающие при работе с параметрами фигур или выходе за границы экрана.

2. Модификация класса *diagonal_cross*. В пользовательский класс фигуры *diagonal_cross* были встроены проверки:

- на допустимость размера в конструкторе (если размер ≤ 0 — выбрасывается *NegativeSizeException*);
- на корректность масштабирования (*resize()*);
- на допустимость координат при отрисовке (*draw()*), с генерацией *OutOfScreenException* при выходе за границы экрана.

3. Индивидуальная обработка ошибок при создании объектов. Каждое создание фигуры типа *diagonal_cross* обёрнуто в отдельный блок *try-catch*, что позволяет программе продолжать выполнение даже в случае ошибки и точно указывать, какая фигура не была создана и почему.

4. Безопасное обновление экрана через *safe_shape_refresh()*. Вместо стандартной функции отрисовки *shape_refresh()* была реализована функция *safe_shape_refresh()*, в которой отрисовка каждой фигуры проводится с индивидуальным перехватом исключений. Если при отрисовке фигура вызывает ошибку, она автоматически удаляется из списка *shape::shapes*, а программа продолжает работу.

5. Дополнительные эксперименты с ошибками масштабирования и выхода за экран. В *main()* добавлены вызовы, демонстрирующие:

- ошибку при создании с недопустимым размером (*bad_cross1*);
- ошибку из-за слишком большого размера (*bad_cross2*);
- ошибку при масштабировании с отрицательным и чрезмерным коэффициентом (*bad_cross3*);
- искусственный выход за границы экрана методом *move()* (*bad_cross4*).

6. Минимизация влияния ошибок на итоговую картинку. Несмотря на наличие ошибок при создании или трансформации фигур, программа успешно продолжает выполнение, отображая корректно созданные фигуры и выдавая сообщения об ошибках, что позволяет анализировать поведение системы без её остановки.

Описание классов исключений

Для реализации контроля над возможными ошибками в программе были разработаны и использованы собственные классы исключений.

Класс *NegativeSizeException* используется для обработки ошибок, связанных с недопустимыми значениями параметров, определяющих размер фигуры. Исключение генерируется, например, при создании фигуры с

нулевым или отрицательным размером или масштабировании фигуры с использованием отрицательного коэффициента.

Класс *OutOfScreenException* применяется для выявления ситуации, при которой хотя бы одна из точек фигуры выходит за пределы границ экрана. Это может произойти при рисовании фигуры, перемещении или после масштабирования. Проверка осуществляется в методе *draw()* с использованием функции *on_screen()*.

Обоснование набора и вида классов исключений

Выбор набора исключений основан на анализе двух основных типов потенциальных ошибок:

- ошибки, связанные с некорректными параметрами, передаваемыми при создании или масштабировании фигур (например, нулевой или отрицательный размер);
- ошибки, возникающие при выходе фигуры за пределы экрана во время отрисовки.

Для обработки этих ситуаций в коде были введены два отдельных класса исключений, которые наследуются напрямую от *std::exception*. Каждый из них содержит собственное сообщение об ошибке, доступное через метод *what()*.

Такой подход позволяет:

- точно различать причины сбоев;
- обрабатывать ошибки как индивидуально, так и обобщённо через базовый класс *std::exception*;
- повысить читаемость и отладочность кода;
- обеспечить простое масштабирование — можно легко добавить новые типы исключений при необходимости.

Обоснование размещения операторов throw

Операторы *throw* размещены в тех точках программы, где возможно возникновение исключительной ситуации:

- в конструкторе — для проверки допустимости параметров;

- в методах *resize()* — для контроля масштабирования;
- в методе *draw()* — для контроля положения фигуры на экране.

Это позволяет как можно раньше зафиксировать ошибку и передать управление в блок обработки на более высоком уровне, не допуская повреждения общего состояния программы.

Обоснование размещения блоков *try-catch*

Блоки *try-catch* размещены вокруг каждого потенциально опасного действия, что позволяет:

- локализовать обработку ошибок;
- избежать аварийного завершения программы;
- продолжить выполнение с корректными объектами.

Основные места размещения:

1. Создание объектов типа *diagonal_cross* Каждое создание фигуры (*left_ear*, *right_ear*, *tie*, *bad_cross1* и др.) заключено в отдельный блок *try-catch*.

Это позволяет:

- точно определить, какой объект не удалось создать;
- вывести индивидуальное сообщение об ошибке;
- не добавлять сбойную фигуру в общую коллекцию.

2. Масштабирование фигур (*resize*) Операции изменения размера, потенциально вызывающие исключения (*NegativeSizeException*), обёрнуты в блоки *try-catch*. Это позволяет сохранить рабочее состояние программы даже при ошибке масштабирования.

3. Безопасная отрисовка (*safe_shape_refresh*)

Вместо общей отрисовки всех фигур был реализован специальный метод *safe_shape_refresh()*, в котором:

- каждая фигура отрисовывается в своём *try*-блоке;
- при возникновении ошибки она удаляется из списка фигур;
- программа продолжает отрисовку оставшихся объектов.

Такой подход обеспечивает:

- гибкий контроль за выполнением каждого действия;
- устойчивость программы к сбоям отдельных объектов;
- минимизацию искажений итогового изображения.

Благодаря распределённой структуре обработки ошибок исключения не блокируют выполнение всей программы, а сообщаются пользователю в виде понятных сообщений.

Текст программы

```
#include <iostream>
#include <exception>
#include <string>
#include "screen.h"
#include "shape.h"

class NegativeSizeException : public std::exception {
    std::string msg;

public:
    explicit NegativeSizeException(const std::string &m) : msg(m) {
    }

    const char *what() const noexcept override {
        return msg.c_str();
    }
};

class OutOfScreenException : public std::exception {
    std::string msg;

public:
    explicit OutOfScreenException(const std::string &m) : msg(m) {
    }

    const char *what() const noexcept override {
        return msg.c_str();
    }
};

class diagonal_cross : public shape {
protected:
    point center;
    int size;

public:
    diagonal_cross(point c, int s) : center(c), size(s) {
        if (size <= 0) {
            throw NegativeSizeException("diagonal_cross: size must be positive");
        }
    }

    point north() const override { return point(center.x, center.y + size); }
    point south() const override { return point(center.x, center.y - size); }
    point east() const override { return point(center.x + size, center.y); }
    point west() const override { return point(center.x - size, center.y); }
```

```

point neast() const override { return point(center.x + size, center.y - size); }
point seast() const override { return point(center.x + size, center.y + size); }
point nwest() const override { return point(center.x - size, center.y - size); }
point swest() const override { return point(center.x - size, center.y + size); }

void draw() override {
    if (!on_screen(nwest().x, nwest().y) ||
        !on_screen(neast().x, neast().y) ||
        !on_screen(seast().x, seast().y) ||
        !on_screen(swest().x, swest().y)) {
        throw OutOfScreenException("diagonal_cross: figure is out of screen
bounds");
    }
    put_line(nwest(), seast());
    put_line(neast(), swest());
}

void move(int dx, int dy) override {
    center.x += dx;
    center.y += dy;
}

void resize(double factor) override {
    if (factor <= 0) {
        throw NegativeSizeException("diagonal_cross: resize factor must be > 0");
    }
    size = static_cast<int>(size * factor);
}
};

void up(shape &p, const shape &q) {
    point n = q.north();
    point s = p.south();
    p.move(n.x - s.x, n.y - s.y + 1);
}

void down(shape &p, const shape &q) {
    point s = q.south();
    point n = p.north();
    p.move(s.x - n.x, s.y - n.y - 1);
}

void left(shape &p, const shape &q) {
    point w = q.west();
    point e = p.east();
    p.move(w.x - e.x - 1, w.y - e.y);
}

void right(shape &p, const shape &q) {
    point e = q.east();
    point w = p.west();
    p.move(e.x - w.x + 1, e.y - w.y);
}

static void safe_shape_refresh() {
    screen_clear();
    for (auto it = shape::shapes.begin(); it != shape::shapes.end();) {
        shape *current = *it;
        try {
            current->draw();
            ++it;
        }
    }
}

```

```

    } catch (std::exception &ex) {
        std::cout << "[Draw Error] " << ex.what() << std::endl;
        auto next = it;
        ++next;
        delete current;
        it = next;
    }
    catch (...) {
        std::cout << "[Unknown Draw Error]" << std::endl;
        auto next = it;
        ++next;
        delete current;
        it = next;
    }
}
screen_refresh();
}

int main() {
#ifdef LOCAL
    freopen("output.out", "w", stdout);
#endif
    screen_init();

    rectangle hat(point(0, 0), point(14, 5));
    rectangle face(point(16, 0), point(28, 8));
    line brim(point(20, 10), 17);

    diagonal_cross *left_ear = nullptr;
    diagonal_cross *right_ear = nullptr;
    diagonal_cross *tie = nullptr;
    diagonal_cross *bad_cross1 = nullptr;
    diagonal_cross *bad_cross2 = nullptr;
    diagonal_cross *bad_cross3 = nullptr;
    diagonal_cross *bad_cross4 = nullptr;

    try {
        left_ear = new diagonal_cross(point(5, 15), 2);
    } catch (const std::exception &ex) {
        std::cout << "[Creation Error] left_ear: " << ex.what() << std::endl;
    }
    try {
        right_ear = new diagonal_cross(point(12, 15), 2);
    } catch (const std::exception &ex) {
        std::cout << "[Creation Error] right_ear: " << ex.what() << std::endl;
    }
    try {
        tie = new diagonal_cross(point(22, 15), 3);
    } catch (const std::exception &ex) {
        std::cout << "[Creation Error] tie: " << ex.what() << std::endl;
    }
    try {
        bad_cross1 = new diagonal_cross(point(5, 5), -4);
    } catch (const std::exception &ex) {
        std::cout << "[Creation Error] bad_cross1: " << ex.what() << std::endl;
    }
    try {
        bad_cross2 = new diagonal_cross(point(2, 2), 9999);
    } catch (const std::exception &ex) {
        std::cout << "[Creation Error] bad_cross2: " << ex.what() << std::endl;
    }
}

```

```

    try {
        bad_cross3 = new diagonal_cross(point(5, 20), 2);
    } catch (const std::exception &ex) {
        std::cout << "[Creation Error] bad_cross3: " << ex.what() << std::endl;
    }
    try {
        bad_cross4 = new diagonal_cross(point(10, 20), 2);
    } catch (const std::exception &ex) {
        std::cout << "[Creation Error] bad_cross4: " << ex.what() << std::endl;
    }

    safe_shape_refresh();
    std::cout << "=== Generated... ===\n";

    hat.rotate_right();
    brim.resize(2.0);
    face.resize(1.2);

    try {
        if (bad_cross3) {
            bad_cross3->resize(-3.0);
        }
    } catch (const std::exception &ex) {
        std::cout << "[Resize Error] bad_cross3: " << ex.what() << "\n";
    }
    try {
        if (bad_cross3) {
            bad_cross3->resize(100);
        }
    } catch (const std::exception &ex) {
        std::cout << "[Resize Error] bad_cross3: " << ex.what() << "\n";
    }

    safe_shape_refresh();
    std::cout << "=== Prepared... ===\n";

    face.move(-3, 10);
    up(brim, face);
    up(hat, brim);
    if (tie) down(*tie, face);
    if (left_ear) left(*left_ear, face);
    if (right_ear) right(*right_ear, face);
    if (bad_cross4) bad_cross4->move(100, 100);
    safe_shape_refresh();
    std::cout << "=== Ready! ===\n";

    screen_destroy();
    return 0;
}

```

Результаты работы программы

Программа работает корректно и выводит отлавливаемые ошибки.

Результат выполнения программы представлен на рисунке 1.

[illegible]

Рисунок 1 – Вывод программы.

Выводы

В ходе выполнения работы были изучены и реализованы принципы обработки исключительных ситуаций с применением к библиотеке графических фигур. Программа была доработана с учётом безопасного выполнения: потенциально аварийные места в коде были усилены с помощью механизмов *throw* и *try-catch*.

Создание собственных классов исключений позволило:

- точно классифицировать ошибки (логические, связанные с параметрами, и визуальные, связанные с отображением);
- централизованно обрабатывать исключения на верхнем уровне программы;
- обеспечить вывод содержательных сообщений пользователю без аварийного завершения.

Размещение блоков обработки исключений на уровне *main()* позволило минимизировать искажения итогового изображения и сохранить контроль над выполнением программы при возникновении ошибок. Реализация обработки исключений повысила устойчивость программы, улучшила её отладку и сделала поведение в нестандартных ситуациях предсказуемым и управляемым.

Список используемых источников

1. Колинко П. Г. Пользовательские контейнеры: учебно-метод. пособие. СПб.: СПбГЭТУ «ЛЭТИ», 2025. 64 с. (вып.2502)